PhotoMatch — Manual Técnico de Defensa

Trabajo Práctico 06: Continuous Delivery (CD) y Ambientes Aislados

Ingeniería de Software III · Universidad Católica de Córdoba (UCC) · 2026

Alumno: Lautaro Alejo Vallino
Legajo: 2301009 · Comisión 2
Aplicación: PhotoMatch (Node.js 22 / Express / PostgreSQL 16)

Release Oficial: `v6.0.0`
Ambientes: QA (:3001) y PROD (:3002)
Pipeline: GitHub Actions + Self-Hosted Runner

A. Introducción Conceptual

1. Continuous Delivery vs. Continuous Deployment

Uno de los conceptos nucleares de la materia es distinguir con precisión qué separa a la Entrega Continua del Despliegue Continuo:

- **Integración Continua (CI):** Práctica en la que cada desarrollador integra código frecuentemente a una rama troncal (`main`). Cada commit dispara validación estática, compilación, ejecución de tests unitarios y cálculo de cobertura.
- **Entrega Continua (Continuous Delivery - CD):** Extensión de la CI donde cada cambio integrado que pasa las pruebas se empaqueta de forma inmutable y queda **listo para ser puesto en producción en cualquier momento**. El paso final hacia producción está protegido por una **decisión humana o gate de negocio** (aprobación manual).
- **Despliegue Continuo (Continuous Deployment):** Todo cambio que supera la batería automatizada de pruebas se promueve **automáticamente a producción sin intervención humana**.

En PhotoMatch implementamos estrictamente **Continuous Delivery**: el pasaje al ambiente de QA es automático tras pasar CI y publicación, pero la promoción a Producción requiere la aprobación explícita del responsable técnico mediante las **Environment Protection Rules** de GitHub.

2. Principio «Build Once, Deploy Many»

Un anti-patrón clásico en pipelines iniciales es compilar o construir la imagen Docker en QA, y luego volver a ejecutar `docker build` cuando se decide desplegar a Producción.

> **Riesgo de Recompilación**
>
> Entre una construcción y otra pueden cambiar dependencias remotas (`npm install`), hashes de paquetes base del sistema operativo o variables de tiempo de compilación. El artefacto probado en QA **no sería el mismo** que se ejecuta en Producción.

En PhotoMatch respetamos el principio a ultranza:

1. `build-backend` y `build-frontend` construyen las imágenes Docker una única vez y las empaquetan en archivos `.tar` temporales como artefactos de GitHub Actions.
2. El job `publish` toma exactamente esos archivos `.tar`, los carga en el daemon y los publica en GitHub Packages (GHCR) con la etiqueta inmutable `sha-<COMMIT_SHA>`.
3. Los jobs `deploy-qa` y `deploy-prod` consumen **exactamente el mismo tag SHA de GHCR**, garantizando paridad binaria total entre ambos ambientes.

3. Aislamiento de Ambientes: QA vs. Producción Tener ambientes aislados previene que pruebas funcionales o errores de desarrollo impacten en los datos reales de los clientes:

- **Redes aisladas:** Contenedores en redes Docker independientes (`photomatch-qa-net` vs `photomatch-prod-net`).
- **Persistencia separada:** Volúmenes de base de datos PostgreSQL independientes (`photomatch-qa-db-data` vs `photomatch-prod-db-data`).
- **Puertos de acceso diferenciados:** QA atiende en `http://127.0.0.1:3001` y Producción en `http://127.0.0.1:3002`.
- **Secretos independientes:** `SESSION_SECRET`, `POSTGRES_PASSWORD` y credenciales gestionadas en GitHub Environments con alcance restringido.

B. Estado Inicial de PhotoMatch antes del TP6

Al finalizar el TP5, PhotoMatch contaba con:

- Una suite de pruebas unitarias con umbral de cobertura estricto (≥80% líneas, ≥70% ramas).
- Un workflow de CI (`ci.yml`) limitado a ejecutar `npm test` sobre el backend.
- Sin contenedorización formal del frontend (Nginx no existía en el repositorio).
- Sin publicación de artefactos en registros de contenedores (GHCR).
- Sin pipeline de despliegue automatizado ni concepto formal de ambientes QA y Producción.

C. Implementación Paso a Paso

1. **Contenedorización:** Se creó el `Dockerfile` de producción del frontend basado en `nginx:alpine` y `frontend/default.conf.template` como reverse proxy dinámico hacia Express.
2. **Parametrización del Backend:** El `Dockerfile` del backend incorpora el build-arg `COMMIT_SHA` y lo expone en `/health`.
3. **Pipeline Desacoplado en 5 Etapas:**
   - `build-backend`: Corre tests, evalúa cobertura con `c8`, construye la imagen y exporta `.tar`.
   - `build-frontend`: Construye la imagen Nginx y exporta `.tar`.
   - `publish`: Corre solo en `main` tras ambos builds. Publica los `.tar` en GHCR con etiqueta `sha-${{ github.sha }}`.
   - `deploy-qa`: Despliega automáticamente en QA (`:3001`) y ejecuta `smoke.sh`.
   - `deploy-prod`: Requiere aprobación humana (`environment: production`), despliega en Producción (`:3002`) y corre `smoke.sh`.
4. **Resolución del Desafío de macOS Keychain:** El runner corre como servicio `launchd`. Para evitar bloqueos interactivos con el llavero del sistema operativo, `scripts/deploy-local.sh` utiliza `DOCKER_CONFIG` temporal anónimo para paquetes públicos y reutiliza la imagen precargada `postgres:16-alpine`.
5. **Optimización de Caché npm:** Se eliminó la directiva `cache: npm` en los jobs del runner propio para evitar la subida redundante de 687 MB por corrida.

D. Decisiones de Ingeniería y Trade-offs

Decisión Tomada	Alternativa Descartada	Justificación Técnica y Trade-off
Fallback Local con Self-Hosted Runner	Planes gratuitos de Render / Neon	Render suspende contenedores (demoras de 50s); Neon restringe conexiones. El runner local garantiza latencia cero y disponibilidad inmediata.

Decisión Tomada	Alternativa Descartada	Justificación Técnica y Trade-off
Separación Build y Publish vía `.tar`	Construir y pushear en el mismo job	Garantiza que nada se publique si fallan las pruebas unitarias. Costo: transferencia de artefactos en GitHub (20s).
Gate de Aprobación Humana en PROD	Despliegue Continuo automático a PROD	Cumple el requisito contractual de CD: el pasaje a producción es una decisión de negocio auditada.
Rollback por Recreación en Compose	Blue/Green con balanceador activo	Blue/Green duplica consumo de RAM. El retorno al SHA anterior se midió en 25,306 s hasta `/health` con DB y SHA; las imágenes estaban cacheadas, así que no es un SLA.

E. Pruebas y Evidencias Reales

Evidencias Auditables en GitHub

1. **Gate de Producción Rechazado:** CI Run 40277582164. Motivo registrado: *«QA, integración y E2E pasan, pero se difiere la promoción hasta revisar rollback y observabilidad de producción»*. Producción permaneció intacta.
2. **Gate de Producción Aprobado:** CI Run 40284305415. Aprobado por `LautyUCC`. Promoción exitosa a Producción en commit `1caea7c`.
3. **Rollback PROD medido:** `1caea7c` a `99f83ef`, **25,306 s** desde antes del pull hasta `/health` con DB y SHA; restauración **25,278 s**. Mismo contenedor PostgreSQL. Rastro en `docs/defensa/evidencias/rollback-prod-2026-10-26.log`. Los 7,42 s anteriores solo medían recreación parcial.
4. **Release Oficial:** `v6.0.0` publicada en GitHub con notas y tag inmutable.

F. Preparación para la Defensa Oral

Pregunta 1: ¿Por qué las imágenes se construyen en el job de build y se publican en un job separado?

Respuesta para defender:

Para respetar el principio de fail-fast y no ensuciar el registro de contenedores. Si los tests unitarios o el umbral de cobertura fallan, no debemos publicar un artefacto defectuoso. `publish` solo corre si `build-backend` y `build-frontend` completaron con éxito.

Pregunta 2: ¿Cómo garantizan que la versión desplegada en Producción sea idéntica a la probada en QA?

Respuesta para defender:

Mediante el principio «Build Once, Deploy Many» e inmutabilidad en GHCR. La imagen se etiqueta con el hash del commit (`sha-<COMMIT>`). En QA y en PROD se despliega exactamente el mismo digest; nunca se vuelve a ejecutar `docker build`.

Pregunta 3: ¿Qué sucede si el smoke test falla después del despliegue?

Respuesta para defender:

`scripts/smoke.sh` retorna código de salida 1, haciendo fallar el paso de GitHub Actions. Esto alerta inmediatamente al equipo, bloquea el workflow y permite iniciar el rollback al SHA previo.

Pregunta 4 (Difícil): ¿Qué sucedería con la base de datos de producción durante un rollback si el commit revertido aplicó una migración destructiva?

Respuesta para defender:

Un rollback de contenedores solo revierte el código de la aplicación, no el estado de los datos. Para mitigar esto, se debe seguir el patrón de migraciones expand/contract (hacer cambios retro-compatibles en dos fases). En PhotoMatch, el volumen de datos es persistente y externo al ciclo de vida del contenedor, por lo que los datos no se destruyen al recrear la aplicación.

G. Resumen Final de Estudio

- **Continuous Delivery:** Automatización completa con gate de aprobación humana hacia producción.
- **Inmutabilidad:** Imágenes etiquetadas por SHA construidas una sola vez.
- **Aislamiento:** QA (:3001) y PROD (:3002) con redes, volúmenes y puertos disjuntos.
- **Pitch de 2 minutos:** *«En TP6 construimos imágenes una sola vez, las publicamos por SHA y desplegamos QA automáticamente en Docker local. PROD requiere aprobación humana; mostramos un rechazo con motivo y una aprobación. El rollback real de PROD al SHA anterior tardó 25,306 s hasta salud HTTP con DB conectada; restaurarlo tomó 25,278 s, conservando PostgreSQL.»*